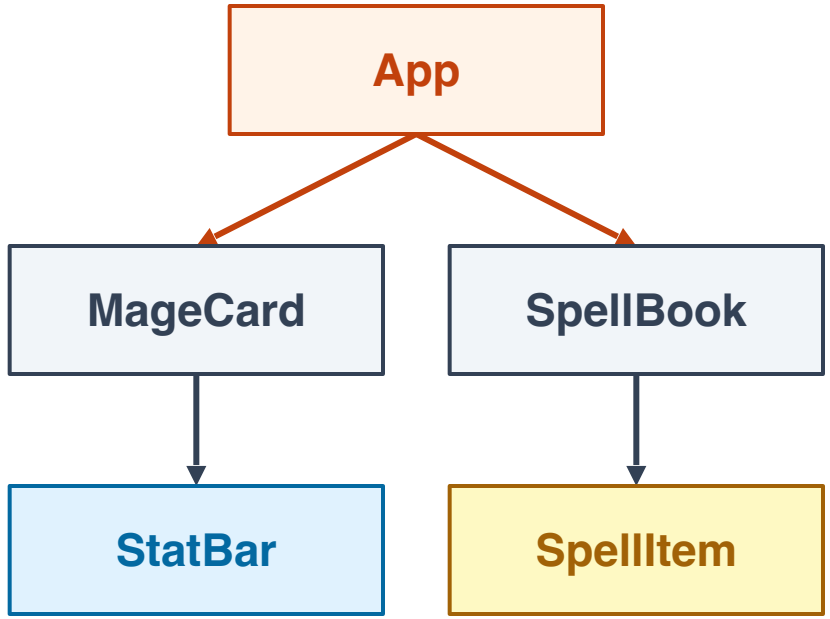
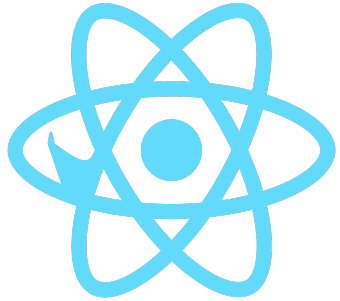
React

Компоненты, состояние и события

Frontend-разработчик





Что будем уметь

- Создавать проект React на Vite и читать его структуру
- Описывать интерфейс компонентами и JSX
- Передавать данные через props, а события — через колбэки
- Хранить состояние в useState и обновлять его без мутаций
- Объяснять, что такое рендер и почему состояние — снимок
- Решать, где хранить состояние, а что вычислять при рендере

Демонстрация после лекции: «Жёлтый маг» на React

Где мы сейчас

Тема	Что уже знаем	Что добавим
04 JavaScript	функции, объекты, map и filter	компоненты и обновления без мутаций
05 Интерфейс	f(state) → DOM, сравнение описаний, ключи	как это устроено в React
06 Сборка	Vite, npm, ESLint	проект React на Vite

Сегодня — модель из темы 05, но DOM обновляет библиотека

Что такое React

- Библиотека для описания интерфейса из компонентов
- Сама сравнивает описания экрана и меняет DOM точечно
- Пакет react — компоненты и хуки, react-dom — работа с DOM браузера
- Маршруты, загрузку данных и стили решают другие инструменты

React — библиотека интерфейса, а не весь фреймворк приложения

Создаём проект

```
$ npm create vite@latest yellow-mage -- --template react
$ cd yellow-mage
$ npm install
$ npm run dev

  ➜  Local:   http://localhost:5173/
```

Тот же Vite из прошлой лекции.

Шаблон добавляет react, react-dom и плагин для JSX.

React-проект — обычный проект Vite с плагином для JSX

Что внутри проекта

Файл	Зачем
index.html	пустой и подключение main.jsx
src/main.jsx	точка входа: монтирует App в #root
src/App.jsx	корневой компонент
vite.config.js	подключает плагин @vitejs/plugin-react
package.json	react, react-dom, vite и npm scripts

В HTML один пустой div — остальной интерфейс строит React

Точка входа

```
// src/main.jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

createRoot связывает React с элементом DOM.

StrictMode включает дополнительные проверки в режиме разработки.

Всем, что внутри #root, управляет React

Первый компонент

```
// src/App.jsx
function App() {
  return <h1>Жёлтый маг</h1>;
}

export default App;
```

Компонент — функция с именем с большой буквы.

Она возвращает описание того, что показать.

Компонент — функция, которая возвращает разметку

JSX — это JavaScript

```
// пишем
<button disabled={false}>Искра (5)</button>

// сборщик превращает в вызов
jsx("button", { disabled: false, children: "Искра (5)" })

// вызов возвращает обычный объект
{ type: "button",
  props: { disabled: false, children: "Искра (5)" } }
```

Тот же vnode из темы 05.

JSX не HTML: это синтаксис, который убирает сборщик.

JSX превращается в вызовы функций, а они — в обычные объекты

Правила JSX

В HTML	В JSX	Почему
несколько элементов подряд	один корень или <>…</>	функция возвращает одно значение
<img> 	<img /> 	теги закрываются явно
class="card"	className="card"	имя свойства DOM
for="name"	htmlFor="name"	имя свойства DOM
onclick="cast()"	onClick={cast}	функция, а не строка
style="color: red"	style={{ color: "red" }}	объект стилей

JSX ближе к DOM API, чем к HTML

Выражения в фигурных скобках

```
const mage = { name: "Жёлтый маг", hp: 24, maxHp: 30 };

function MageCard() {
  return (
    <section className="mage">
      <h2>{mage.name}</h2>
      <p>HP: {mage.hp} / {mage.maxHp}</p>
      <progress value={mage.hp} max={mage.maxHp} />
    </section>
  );
}
```

В {} — любое выражение JavaScript.

if и for внутри {} писать нельзя: это инструкции.

{} — окно из разметки в JavaScript

Значения вставляются как текст

```
const name = '<img src="x" onerror="alert(1)">';

function Greeting() {
  return <p>Привет, {name}</p>;
}

// на экране — сама строка, тега img нет
```

Как textContent из темы 03.

Строка с разметкой остаётся строкой.

Значения в {} вставляются как текст, а не как HTML

Компонент из компонентов

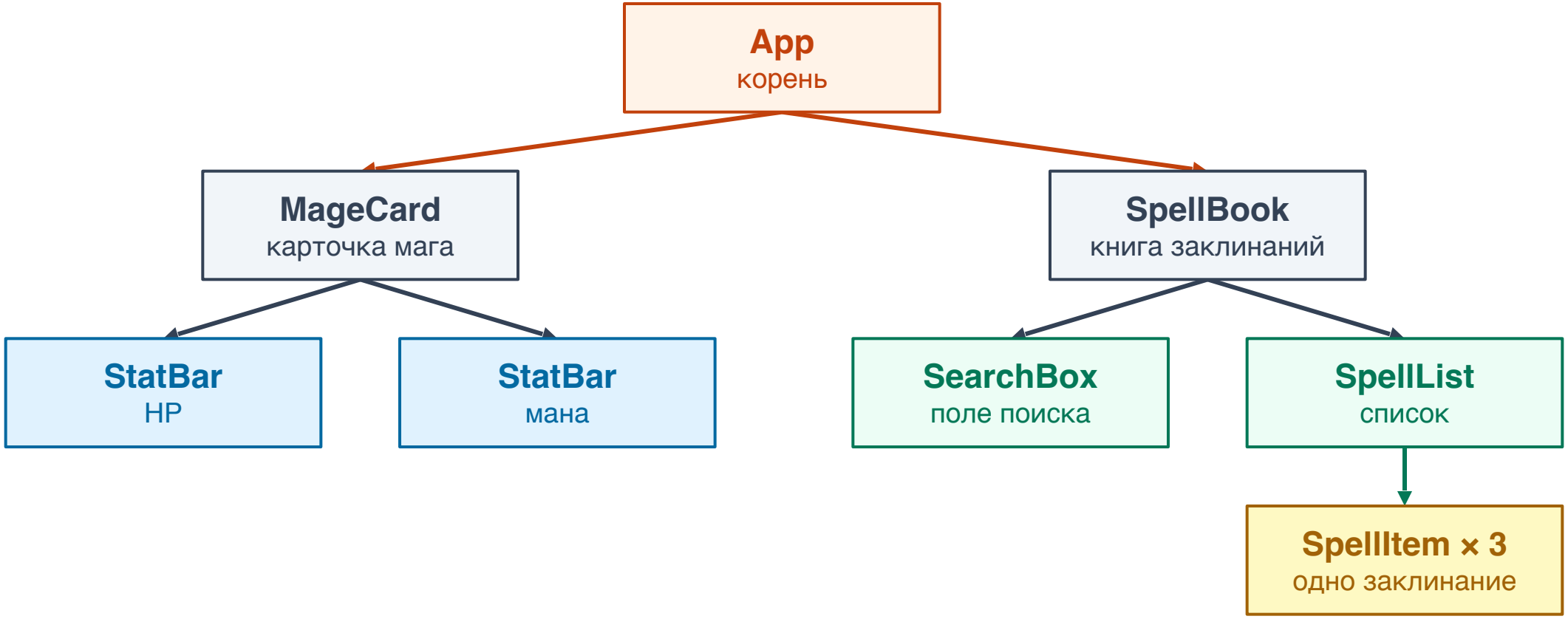
```
function App() {
  return (
    <main>
      <MageCard />
      <SpellBook />
    </main>
  );
}
```

С большой буквы — компонент.

С маленькой — тег DOM: main, p, button.

Интерфейс собирается из компонентов, как функция из функций

Дерево компонентов



Каждый компонент отвечает за свою часть экрана

Props: входные данные

```
function StatBar({ label, value, max }) {
  return (
    <div className="stat">
      {label}: {value} / {max}
      <progress value={value} max={max} />
    </div>
  );
}

<StatBar label="HP" value={24} max={30} />
<StatBar label="Мана" value={20} max={20} />
```

Атрибуты JSX собираются в один объект props.

Деструктуризация — из темы 04.

Props — аргументы функции-компонента

Компонент — чистая функция

```
let renders = 0;

function SpellCount({ spells }) {
  renders++;              // меняет переменную снаружи
  spells.push("Искра");  // меняет чужие данные из props
  return <p>Заклинаний: {spells.length}</p>;
}
```

Компонент не меняет ничего снаружи: ни переменные, ни props.

StrictMode в разработке вызывает его дважды — и ошибка видна сразу.

Те же props → тот же результат, без изменений снаружи

Условный рендеринг

```
function SpellList({ spells }) {
  if (spells.length === 0) {
    return <p>Книга пуста</p>;
  }
  return <ul>{/* список */}</ul>;
}

// внутри JSX
{mana < 5 && <p className="warn">Мало маны</p>}
{hp > 0 ? <MageCard /> : <GameOver />}
```

if, && и тернарный оператор — обычный JavaScript.

Слева от && — boolean: при 0 React нарисует «0».

Условия в JSX — те же выражения JavaScript

Список через map

```
function SpellList({ spells }) {
  return (
    <ul>
      {spells.map((spell) => (
        <li key={spell.id}>
          {spell.name} — {spell.cost} маны
        </li>
      ))}
    </ul>
  );
}
```

map из темы 04: массив данных → массив элементов.

Без key React предупредит в Console.

Список на экране — map по массиву данных

Ключ-индекс путает состояние

```
{spells.map((spell, index) => (
  <li key={index}>
    {spell.name} <input placeholder="заметка" />
  </li>
))}

// ввели: Искра «a», Щит «b», Луч «c»; удалили «Искру»
// key={index}:    Щит «a»   Луч «b»
// key={spell.id}:  Щит «b»   Луч «c»
```

С ключом-индексом React решает, что удалён последний элемент.

Текст в input остаётся на месте — у чужого заклинания.

key — стабильный id из данных, не индекс и не Math.random()

Обработчик события

```
function CastButton({ spell }) {
  function handleClick() {
    console.log("Колдую:", spell.name);
  }
  return (
    <button onClick={handleClick}>
      {spell.name}
    </button>
  );
}
```

onClick вместо addEventListener.

Подписку и отписку React делает сам.

Передаём функцию, а не результат её вызова

Передать или вызвать

```
<button onClick={handleClick}>    // передали функцию
<button onClick={handleClick()}>  // вызвали при рендере
<button onClick={() => cast(spell.id)}> // своя функция
```

Скобки после имени выполняют функцию сразу.

Для аргумента — стрелочная функция.

onClick={f()} выполняет f во время отрисовки, а не по клику

События идут вверх через props

```
function SpellItem({ spell, onCast }) {
  return (
    <li>
      {spell.name}
      <button onClick={() => onCast(spell.id)}>
        Колдовать
      </button>
    </li>
  );
}

<SpellItem spell={spell} onCast={handleCast} />
```

Дочерний сообщает, что произошло. Решает родитель.

Пропсы-события — onЧтоТо, обработчики — handleЧтоТо.

Данные идут вниз через props, события — вверх через колбэки

Почему обычная переменная не работает

```
function ManaCounter() {
  let mana = 20;

  function handleCast() {
    mana -= 5;
    console.log(mana);
  }
  return (
    <button onClick={handleCast}>Мана: {mana}</button>
  );
}
```

Вывод после трёх кликов:
15, 10, 5

На кнопке всё ещё «Мана: 20».

Нужно сохранить значение между рендерами и запросить новый рендер

useState

```
import { useState } from "react";

function ManaCounter() {
  const [mana, setMana] = useState(20);

  function handleCast() {
    setMana(mana - 5);
  }
  return (
    <button onClick={handleCast}>Мана: {mana}</button>
  );
}
```

useState возвращает пару: значение и функцию для его замены.

Эта функция запоминает новое значение и запрашивает рендер.

Состояние — память компонента между рендерами

Правила хуков

```
function MageCard({ mage }) {
  const [open, setOpen] = useState(false); // да

  if (mage.hp === 0) {
    const [x, setX] = useState(0); // нет
  }
  for (const spell of mage.spells) {
    useState(spell); // нет
  }
}
```

React различает состояния по порядку вызова хуков.

Порядок должен совпадать в каждом рендере.

Хуки вызываем только на верхнем уровне компонента

У каждого экземпляра своё состояние

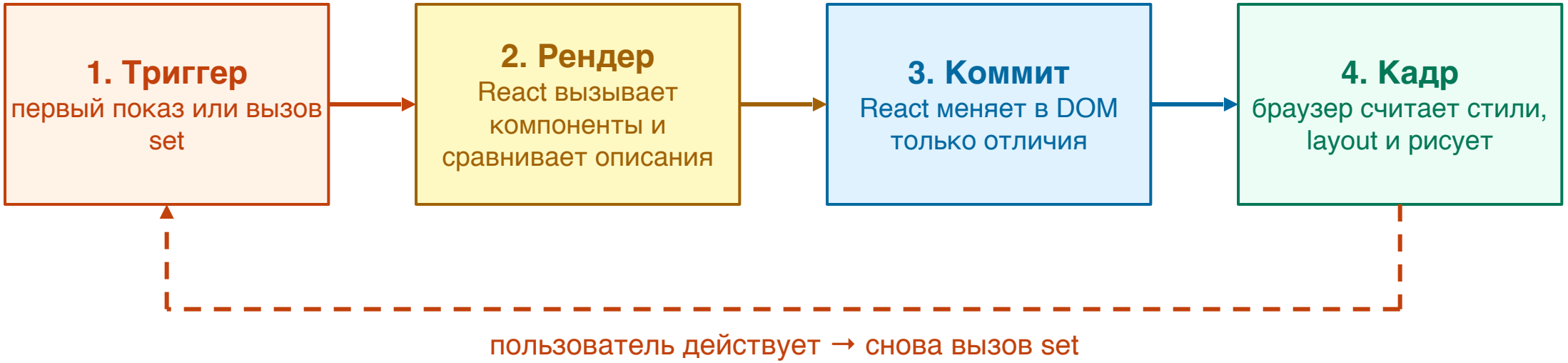
```
function App() {
  return (
    <>
      <ManaCounter />
      <ManaCounter />
    </>
  );
}
```

Клик по первой кнопке не меняет вторую.

Состояние привязано к месту компонента в дереве.

Один компонент — сколько угодно независимых экземпляров

Рендер и коммит



Рендер — вызов ваших функций, коммит — изменение DOM

Рендер не значит изменение DOM

```
function MageCard({ mana }) {
  return (
    <>
      <input placeholder="Заметка о маге" />
      <p>Мана: {mana}</p>
    </>
  );
}

// mana восстанавливается каждую секунду
```

React меняет только текст в <p>.

Текст в поле и фокус остаются: проблема из демонстрации темы 05 решена.

Пишем как полную перерисовку, DOM меняется точечно

Мини-задача: что покажет кнопка

```
function ManaButton() {
  const [mana, setMana] = useState(10);

  function handleClick() {
    setMana(mana - 5);
    setMana(mana - 5);
    console.log(mana);
  }
  return <button onClick={handleClick}>{mana}</button>;
}
```

Кнопку нажали один раз.

Что в Console? Что на кнопке?

Сравните с задачей про rAF из темы 05.

Подумать 30 секунд, ответ на следующем слайде

Разбор: 10 и 5

```
// в этом рендере mana — константа 10
setMana(10 - 5);  // следующий рендер: 5
setMana(10 - 5);  // следующий рендер: всё ещё 5
console.log(10);

// новый рендер: mana === 5
```

set не меняет переменную, а планирует новый рендер.

В теме 05 колбэк читал state.mana в момент вызова. Здесь — значение своего рендера.

Состояние — снимок: обработчик видит значение своего рендера

Функция-обновление

```
function handleClick() {
  setMana((m) => m - 5);
  setMana((m) => m - 5);
}

// очередь: 10 → 5 → 0
```

React применяет функции по очереди к актуальному значению.

Нужно, когда обновлений несколько подряд.

set(x => …) — «возьми актуальное и измени»

Объект в состоянии

```
const [mage, setMage] = useState({ hp: 30, mana: 20 });

// экран не обновится
mage.mana -= 5;
setMage(mage);

// новый объект
setMage({ ...mage, mana: mage.mana - 5 });
```

React сравнивает старое и новое значение по ссылке, как === в теме 05.

Тот же объект — «ничего не изменилось».

Новое состояние — новый объект; держим его плоским

Массивы в состоянии

Действие	Меняет массив (избегаем)	Новый массив (используем)
добавить	push, unshift	[...spells, spell]
удалить	splice, pop, shift	filter
заменить	spells[i] = x	map
сортировать	sort, reverse	toSorted(), [...spells].sort()

Методы, которые меняют массив на месте, в состоянии не используем

Добавить, удалить, изменить

```
function add(name, cost) {
  const spell = { id: crypto.randomUUID(), name, cost };
  setSpells([...spells, spell]);
}

function remove(id) {
  setSpells(spells.filter((s) => s.id !== id));
}

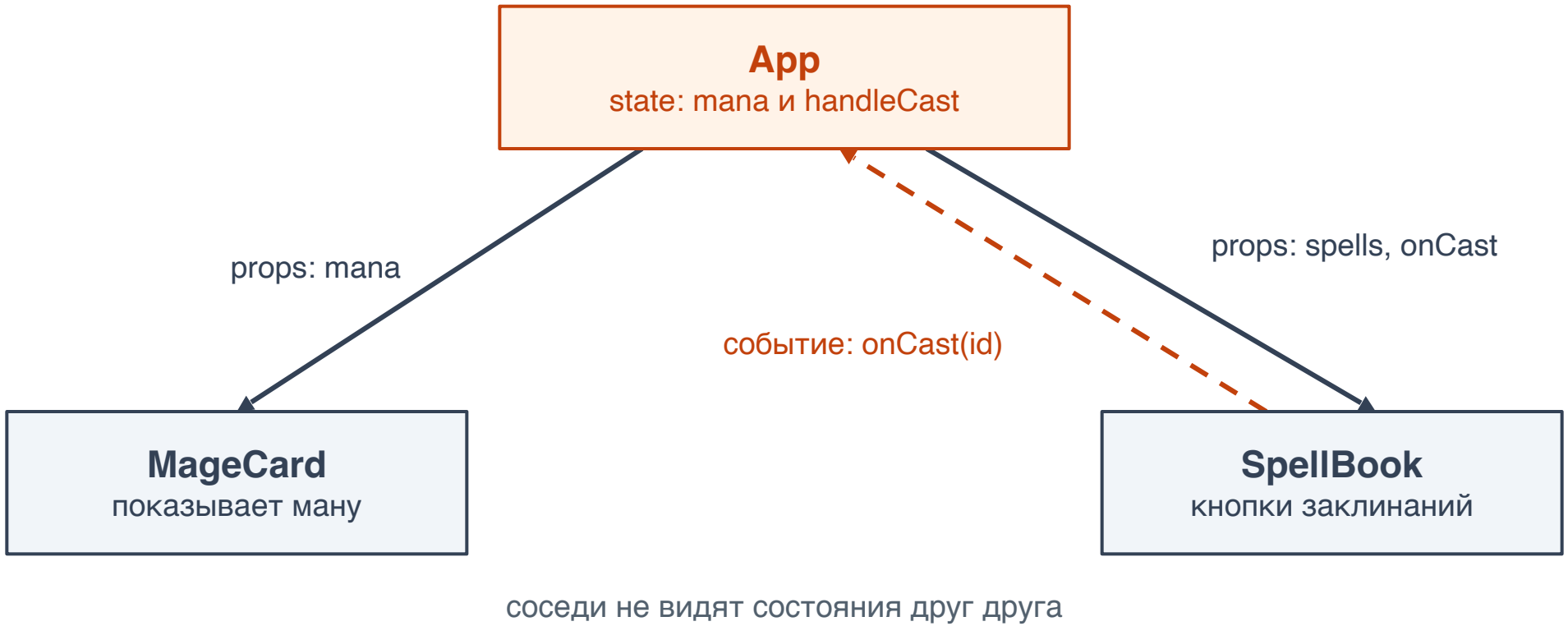
function cheapen(id) {
  setSpells(spells.map((s) =>
    s.id === id ? { ...s, cost: s.cost - 1 } : s,
  ));
}
```

spread, filter и map возвращают новый массив.

Остальные элементы — те же объекты.

filter, map и spread — основные инструменты обновления

Подъём состояния



Общее состояние поднимаем в ближайшего общего родителя

Подъём состояния в коде

```
function App() {
  const [mana, setMana] = useState(20);

  function handleCast(id) {
    const spell = spells.find((s) => s.id === id);
    if (spell.cost <= mana) setMana(mana - spell.cost);
  }
  return (
    <main>
      <MageCard mana={mana} />
      <SpellBook spells={spells} onCast={handleCast} />
    </main>
  );
}
```

App — владелец маны.

MageCard получает число, SpellBook — функцию.

Состояние живёт там, где его используют все, кому оно нужно

Производные данные не храним

```
// избыточно: два состояния про одно и то же
const [spells, setSpells] = useState(initialSpells);
const [count, setCount] = useState(initialSpells.length);

// достаточно
const [spells, setSpells] = useState(initialSpells);
const count = spells.length;
const affordable = spells.filter((s) => s.cost <= mana);
```

Принцип минимального состояния из темы 05.

Вычислять при каждом рендере — нормально.

Что можно вычислить из props и state — вычисляем

Нужно ли это состояние?

Вопрос	Если да
Не меняется со временем?	это не состояние
Приходит от родителя?	это props
Можно вычислить из других данных?	вычисляем при рендере
Ничего из этого	состояние в ближайшем общем родителе

Состояние — только то, что нельзя получить иначе

Как собрать экран на React

Шаг	Что делаем	В «Жёлтом маге»
1	Разбить макет на компоненты	MageCard, SpellBook, SpellItem
2	Сделать статическую версию на props	список из массива, без состояния
3	Найти минимальное состояние	mana, query
4	Решить, где оно живёт	mana — в App, query — в SpellBook
5	Добавить события вверх	onCast, onQueryChange

Сначала структура и данные, потом состояние и события

Управляемое поле

```
function SearchBox({ query, onQueryChange }) {
  return (
    <input
      value={query}
      onChange={(e) => onQueryChange(e.target.value)}
      placeholder="Найти заклинание"
    />
  );
}
```

Значение поля задаёт состояние.

onChange в React срабатывает на каждый ввод, как событие input.

Источник правды для поля — состояние, а не DOM

Поиск без потери фокуса

```
function SpellBook({ spells, onCast }) {
  const [query, setQuery] = useState("");
  const visible = spells.filter((s) =>
    s.name.toLowerCase().includes(query.toLowerCase()),
  );
  return (
    <section>
      <SearchBox query={query} onQueryChange={setQuery} />
      <SpellList spells={visible} onCast={onCast} />
    </section>
  );
}
```

Отфильтрованный список — производное, не состояние.

Фокус в поле не теряется.

Фильтр — вычисление при рендере, а не второе состояние

Форма добавления

```
function AddSpellForm({ onAdd }) {
  const [name, setName] = useState("");
  function handleSubmit(event) {
    event.preventDefault();
    if (name.trim() === "") return;
    onAdd(name.trim());
    setName("");
  }
  return (
    <form onSubmit={handleSubmit}>
      <input value={name} onChange={(e) => setName(e.target.value)} />
      <button type="submit">Добавить</button>
    </form>
  );
}
```

Поле — value и onChange, отправка — onSubmit и preventDefault

React DevTools

Где	На какой вопрос отвечает
Components	какое дерево компонентов, какие у них props и state
Profiler	какие компоненты рендерились и сколько это заняло
Highlight updates	что перерисовывается при действии пользователя

DevTools показывает компоненты, а не только DOM

Что сегодня не трогаем

Тема	Когда
TypeScript для props и состояния	следующая лекция
useEffect, загрузка данных, гонка запросов	React II
useReducer, Context, сброс состояния через key	React II
useMemo, useCallback, memo	обзорно: мемоизацию делает React Compiler
маршрутизация, Next.js	по желанию в итоговом проекте

Сегодня — основа, на которой держится всё остальное

Демонстрация после лекции

- npm create vite@latest yellow-mage -- --template react
- «Жёлтый маг»: MageCard, StatBar, SpellBook, SpellItem
- Каст заклинания: состояние маны поднято в App
- Поиск по книге: управляемое поле, фокус не теряется
- React DevTools: дерево, props, state, подсветка рендеров

Код демонстрации прилагается: папка src для шаблона react

Практика: «Мои задачи» на React

- Разбить по пяти шагам: TaskList, TaskItem, AddTaskForm
- Добавление через форму, как AddSpellForm; пустое название не добавляется
- Отметить выполненной и удалить — без мутаций массива
- Фильтр «все / активные / выполненные» — производное от состояния
- Счётчик оставшихся задач вычисляется, а не хранится
- Ключи — id задачи, не индекс

Сдача — ссылкой на Pull Request, как обычно

Что важно унести с собой

- Компонент — функция от props и state, возвращает описание экрана
- JSX → объекты; React сравнивает их и меняет DOM точечно
- Данные идут вниз через props, события — вверх через колбэки
- Состояние — снимок рендера; set планирует следующий рендер
- Новое состояние — новый объект или массив, без мутаций
- Минимум состояния: производное вычисляем, общее — в общем родителе

Следующая лекция: TypeScript

Источники и дальнейшее чтение

- react.dev Learn: Describing the UI, Adding Interactivity, Managing State
- react.dev: Render and Commit, State as a Snapshot, Queueing a Series of State Updates
- react.dev: Rendering Lists, Preserving and Resetting State, Thinking in React
- react.dev Reference: useState, StrictMode, Rules of Hooks, <input>
- Vite Guide: Scaffolding Your First Vite Project
- React Developer Tools

Ссылки на конкретные разделы находятся в заметках слайдов